

Documento de Arquitectura de Software

(Software Architecture Document - SAD)

Sistema de Gestion Academica

Version 1.0

Mayo 2026

Autores: Carlos Stiven Mora Hoyos
Juan Felipe Rodriguez Galindo

Materia: Informatica

Programa: Maestria en Ciencia de la Computacion

Universidad: Universidad Distrital Francisco Jose de Caldas

Historial de Revisiones

Version	Fecha	Descripcion	Autores
1.0	16 de mayo de 2026	Creacion inicial del documento SAD con todos los diagramas UML, vistas arquitectonicas y especificacion completa del sistema de gestion academica.	Carlos Stiven Mora Hoyos, Juan Felipe Rodriguez Galindo

Índice

Historial de Revisiones	1
1. Introduccion	5
1.1. Proposito	5
1.2. Alcance	5
1.3. Definiciones, Acronimos y Abreviaturas	5
1.4. Referencias	5
1.5. Resumen	6
2. Representacion Arquitectonica	7
2.1. Descripcion General	7
2.2. Diagrama de Componentes	7
2.3. Patrones de Diseno Utilizados	7
3. Objetivos y Restricciones Arquitectonicas	8
3.1. Objetivos Arquitectonicos	8
3.2. Restricciones	8
3.3. Requisitos No Funcionales	8
4. Vista de Casos de Uso	9
4.1. Resumen de Actores	9
4.2. Diagrama General de Casos de Uso	9
4.3. Casos de Uso por Actor	11
4.3.1. Casos de Uso del Administrador	11
4.3.2. Casos de Uso del Profesor	13
4.3.3. Casos de Uso del Estudiante	13
4.4. Detalle de Subsistemas	14
4.4.1. Subsistema de Gestion de Carreras	15
4.4.2. Subsistema de Gestion de Estudiantes	16
4.4.3. Subsistema de Gestion de Asignaturas	17
4.4.4. Subsistema de Gestion de Profesores	18
4.4.5. Subsistema de Carga Academica	19
4.4.6. Subsistema de Inscripciones	20
4.4.7. Subsistema de Registro de Notas	21
4.4.8. Subsistema de Prerrequisitos	22
4.4.9. Subsistema de Gestion de Pensum	23
4.4.10. Subsistema de Consultas y Reportes	24
4.5. Matriz de Trazabilidad de Casos de Uso	25
5. Vista Logica	28
5.1. Descripcion General	28
5.2. Estructura de Paquetes	28
5.3. Descripcion de Clases Principales	28
5.3.1. Capa de Modelo (Model)	28
5.3.2. Capa de Acceso a Datos (DAO)	28
5.3.3. Clase OperationResult	29

5.4. Patrones de Diseno Aplicados	29
5.4.1. Patron Singleton	29
5.4.2. Patron DAO	30
6. Vista de Procesos	31
6.1. Descripcion General	31
6.2. Flujo de Inscripcion de Estudiante	31
6.3. Flujo de Registro de Notas	31
6.4. Flujo de Autenticacion	32
7. Vista de Despliegue	33
7.1. Descripcion de la Infraestructura	33
7.2. Configuracion de Hardware y Software	33
7.3. Diagrama de Despliegue	33
7.4. Configuracion de Red	33
8. Vista de Datos	34
8.1. Modelo Entidad-Relacion	34
8.2. Restricciones de Integridad	34
8.3. Triggers y Stored Procedures	34
8.3.1. Trigger trg_calculo_notas	34
8.3.2. Stored Procedure sp_inscribir_estudiante	34
8.4. Control de Acceso (RBAC)	35
9. Tamano y Rendimiento	36
9.1. Estimacion de Tamano	36
9.2. Requisitos de Rendimiento	36
10. Calidad	37
10.1. Mantenibilidad	37
10.2. Portabilidad	37
10.3. Seguridad	37
10.4. Usabilidad	37
10.5. Fiabilidad	37
11. Anexo:Codigo Fuente PlantUML	38
11.1. Diagrama General de Casos de Uso	38
11.2. Casos de Uso del Administrador	38
11.3. Casos de Uso del Profesor	38
11.4. Casos de Uso del Estudiante	38
11.5. Diagramas de Subsistemas	38
11.6. Matriz de Trazabilidad	38

Índice de figuras

1. Diagrama General de Casos de Uso - Sistema de Gestion Academica	10
2. Casos de Uso del Administrador	12
3. Casos de Uso del Profesor	13
4. Casos de Uso del Estudiante	14

5.	Subsistema de Gestion de Carreras	15
6.	Subsistema de Gestion de Estudiantes	16
7.	Subsistema de Gestion de Asignaturas	17
8.	Subsistema de Gestion de Profesores	18
9.	Subsistema de Carga Academica	19
10.	Subsistema de Inscripciones	20
11.	Subsistema de Registro de Notas	21
12.	Subsistema de Prerrequisitos	22
13.	Subsistema de Gestion de Pensum	23
14.	Subsistema de Consultas y Reportes	24
15.	Matriz de Trazabilidad de Casos de Uso	26

Índice de cuadros

1 Introduccion

1.1 Proposito

Este Documento de Arquitectura de Software (SAD) proporciona una descripcion completa de la arquitectura del **Sistema de Gestion Academica**, incluyendo una vision general de la estructura del sistema, sus componentes, las interacciones entre ellos, y las decisiones de diseno que fundamentan su construccion.

El proposito de este documento es servir como referencia principal para:

- Comunicar la vision arquitectonica a los stakeholders del proyecto.
- Guiar el desarrollo e implementacion del sistema.
- Documentar las decisiones arquitectonicas y sus justificaciones.
- Servir como base para el mantenimiento y la evolucion futura del sistema.

1.2 Alcance

Este documento cubre la arquitectura completa del Sistema de Gestion Academica, un aplicativo desarrollado en Java con interfaz grafica JavaFX y base de datos PostgreSQL. El sistema permite la administracion de carreras, estudiantes, asignaturas, profesores, inscripciones, calificaciones y prerrequisitos académicos.

1.3 Definiciones, Acronimos y Abreviaturas

Termino	Definicion
SAD	Software Architecture Document (Documento de Arquitectura de Software)
MVC	Model-View-Controller (Patron de arquitectura)
DAO	Data Access Object (Objeto de Acceso a Datos)
RBAC	Role-Based Access Control (Control de Acceso Basado en Roles)
SP	Stored Procedure (Procedimiento Almacenado)
JDBC	Java Database Connectivity
CU	Caso de Uso
UML	Unified Modeling Language
ER	Entity-Relationship (Modelo Entidad-Relacion)

1.4 Referencias

1. Documento de Especificacion de Requisitos del Sistema de Gestion Academica.
2. Diagramas UML del proyecto (Casos de Uso, Clases, Secuencia, Actividades).
3. Modelo Entidad-Relacion de la base de datos PostgreSQL.
- 4.Codigo fuente del proyecto en el repositorio Git.

1.5 Resumen

Este documento esta organizado siguiendo las vistas arquitectonicas del modelo 4+1 de Kruchten. Las secciones principales incluyen:

- **Vista de Casos de Uso:** Funcionalidades del sistema desde la perspectiva de los actores.
- **Vista Logica:** Estructura estatica del sistema (clases, paquetes).
- **Vista de Procesos:** Dinamica del sistema (flujos, interacciones).
- **Vista de Despliegue:** Configuracion fisica del sistema.
- **Vista de Datos:** Modelo de persistencia y estructura de la base de datos.

2 Representacion Arquitectonica

2.1 Descripcion General

El Sistema de Gestion Academica sigue el estilo arquitectonico **Modelo-Vista-Controlador (MVC)** con una capa adicional de acceso a datos mediante el patron **Data Access Object (DAO)**. La arquitectura es de tipo **Cliente-Servidor** donde el cliente es una aplicacion de escritorio JavaFX y el servidor es una base de datos PostgreSQL.

La logica de negocio reside principalmente en la base de datos mediante triggers y stored procedures, siguiendo un enfoque *database-driven*. La aplicacion Java actua como un pipe transparente que transporta las operaciones del usuario hacia la base de datos y propaga los mensajes de respuesta.

2.2 Diagrama de Componentes

El sistema se compone de los siguientes elementos principales:

- **Capa de Presentacion (View):** Interfaces graficas desarrolladas en JavaFX que permiten la interaccion con los usuarios.
- **Capa de Control (Controller):** Controladores JavaFX que gestionan los eventos de la interfaz y coordinan las operaciones.
- **Capa de Modelo (Model):** Entidades del dominio que representan los datos del negocio (Carrera, Estudiante, Asignatura, etc.).
- **Capa de Acceso a Datos (DAO):** Objetos que abstraen el acceso a la base de datos y encapsulan los resultados mediante la clase `OperationResult`.
- **Capa de Base de Datos:** PostgreSQL con esquema `gestion_academica`, triggers, stored procedures y control de acceso por roles.

2.3 Patrones de Diseno Utilizados

Patron	Descripcion y Uso en el Sistema
MVC	Separacion de responsabilidades entre Modelo (datos), Vista (interfaz) y Controlador (logica de coordinacion).
DAO	Abstraccion del acceso a datos. Cada entidad tiene su propio DAO que encapsula las operaciones CRUD.
Singleton	Gestion de la conexion a la base de datos mediante un unico punto de acceso.
OperationResult	Encapsulacion del resultado de operaciones, propagando exito/error desde la BD hasta la UI.

3 Objetivos y Restricciones Arquitectonicas

3.1 Objetivos Arquitectonicos

1. **Separacion de responsabilidades:** Separar claramente la presentacion, la logica de negocio y el acceso a datos.
2. **Seguridad por roles:** Implementar control de acceso basado en roles (RBAC) a nivel de base de datos.
3. **Integridad de datos:** Centralizar las validaciones y calculos en la base de datos mediante triggers y stored procedures.
4. **Escalabilidad futura:** Permitir la extension del sistema con nuevos modulos sin afectar los existentes.
5. **Facilidad de mantenimiento:** Documentar y estructurar el codigo para facilitar futuras modificaciones.

3.2 Restricciones

1. **Tecnologias:** El sistema debe desarrollarse en Java 17+ con JavaFX para la interfaz grafica.
2. **Base de datos:** Debe utilizarse PostgreSQL 15+ como gestor de base de datos.
3. **Contenedorizacion:** La base de datos debe ejecutarse en un contenedor Docker.
4. **Logica de negocio:** Toda la logica de validacion y calculo debe residir en la base de datos.
5. **Autenticacion:** Debe usarse el sistema de roles nativo de PostgreSQL (RBAC).

3.3 Requisitos No Funcionales

Categoria	Requisito	Descripcion
Seguridad	Autenticacion	El sistema debe autenticar usuarios mediante roles de PostgreSQL.
Seguridad	Autorizacion	Cada rol (Admin, Profesor, Estudiante) debe tener permisos diferenciados.
Rendimiento	Tiempo de respuesta	Las consultas basicas deben responder en menos de 2 segundos.
Usabilidad	Interfaz intuitiva	La interfaz JavaFX debe ser clara y facil de usar.
Mantenibilidad	Codigo modular	El codigo debe seguir patrones de diseno establecidos.

4 Vista de Casos de Uso

4.1 Resumen de Actores

El sistema cuenta con tres actores principales que interactuan con las funcionalidades del sistema segun sus roles:

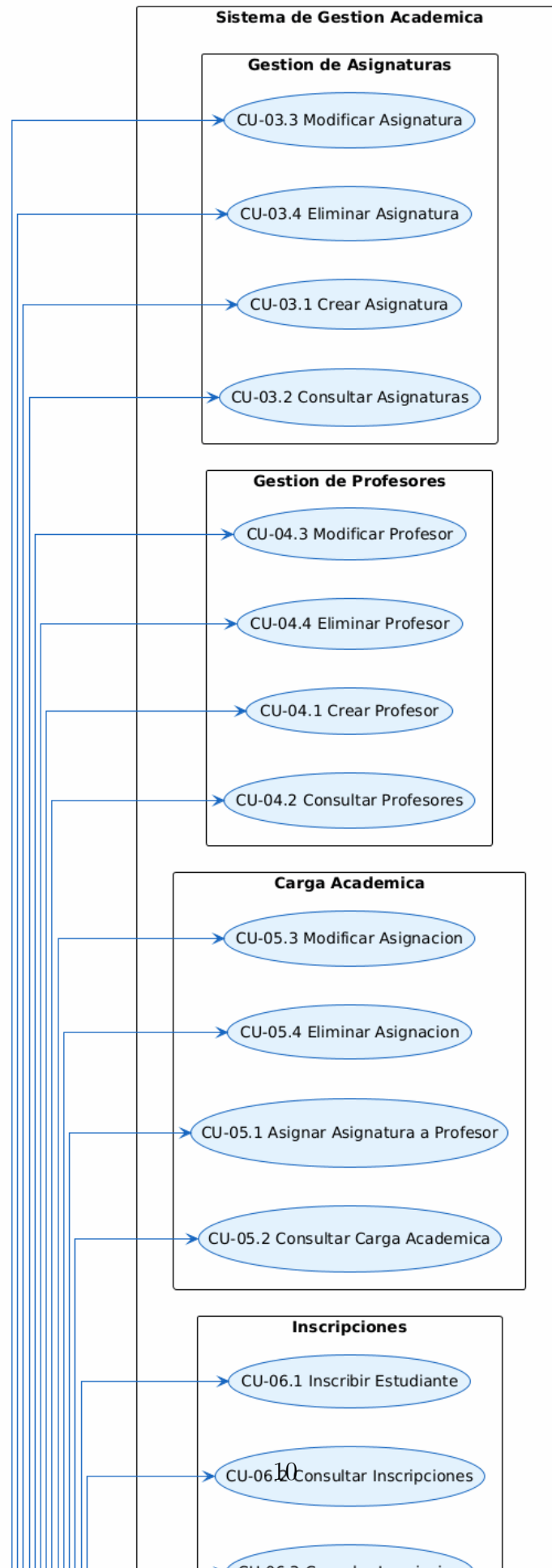
Actor	Tipo	Descripcion
Administrador	Humano primario	Usuario con acceso total al sistema. Gestiona carreras, estudiantes, asignaturas, profesores, inscripciones, notas, prerrequisitos y pensum.
Profesor	Humano primario	Usuario con acceso limitado. Registra notas y consulta informacion academica.
Estudiante	Humano primario	Usuario con acceso de solo lectura. Consulta su informacion academica, pensum y notas.

4.2 Diagrama General de Casos de Uso

La figura 1 presenta el diagrama general de casos de uso del sistema, mostrando todos los actores y los casos de uso desagregados por subsistema.

Please use 'option handwritten true' to enable handwritten

Diagrama General de Casos de Uso - Sistema de Gestion Academica



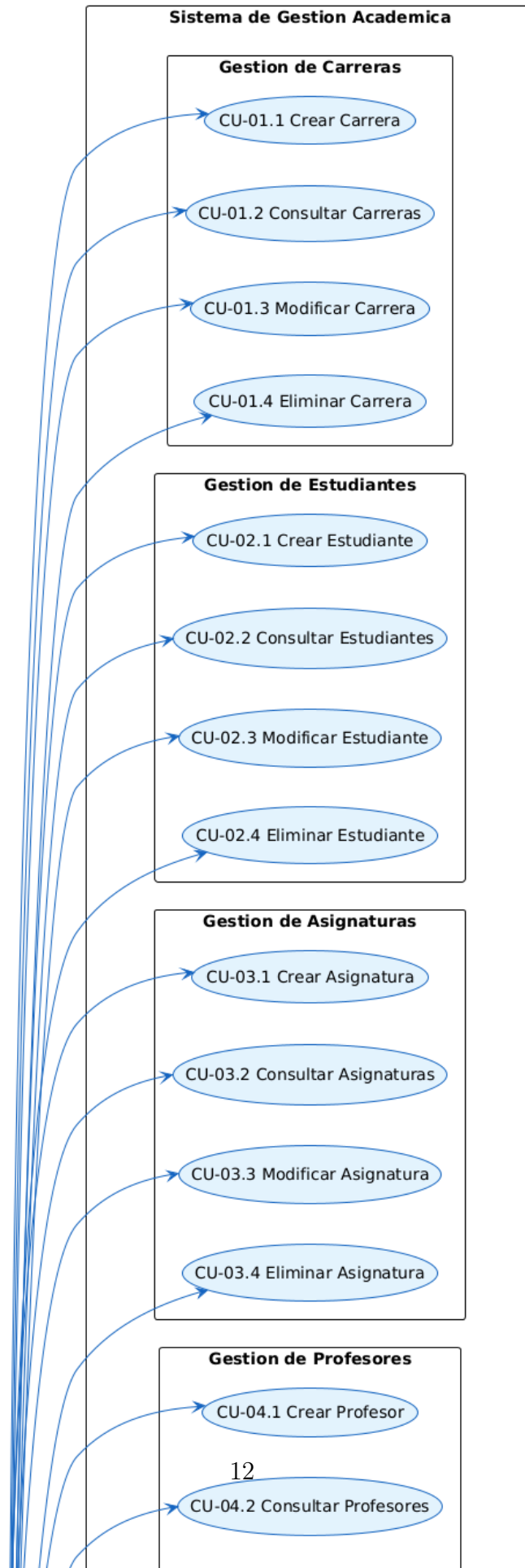
4.3 Casos de Uso por Actor

4.3.1 Casos de Uso del Administrador

El administrador tiene acceso total a todas las funcionalidades del sistema. La figura 2 detalla los casos de uso especificos del administrador.

Please use 'option handwritten true' to enable handwritten

Casos de Uso del Administrador - Sistema de Gestion Academica



4.3.2 Casos de Uso del Profesor

El profesor tiene acceso limitado para registrar notas y consultar informacion academica. La figura 3 muestra sus casos de uso.

Please use '!option handwritten true' to enable handwritten

Casos de Uso del Profesor - Sistema de Gestion Academica

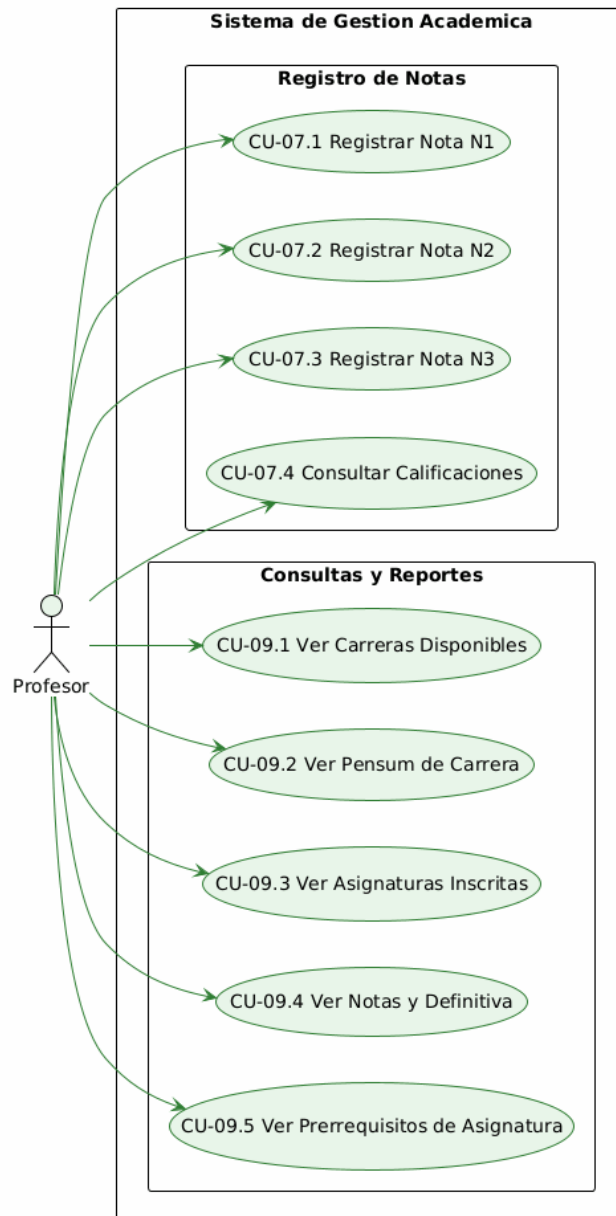


Figura 3: Casos de Uso del Profesor

4.3.3 Casos de Uso del Estudiante

El estudiante solo puede consultar informacion relacionada con su situacion academica. La figura 4 presenta sus casos de uso.

Please use 'option handwritten true' to enable handwritten

Casos de Uso del Estudiante - Sistema de Gestion Academica

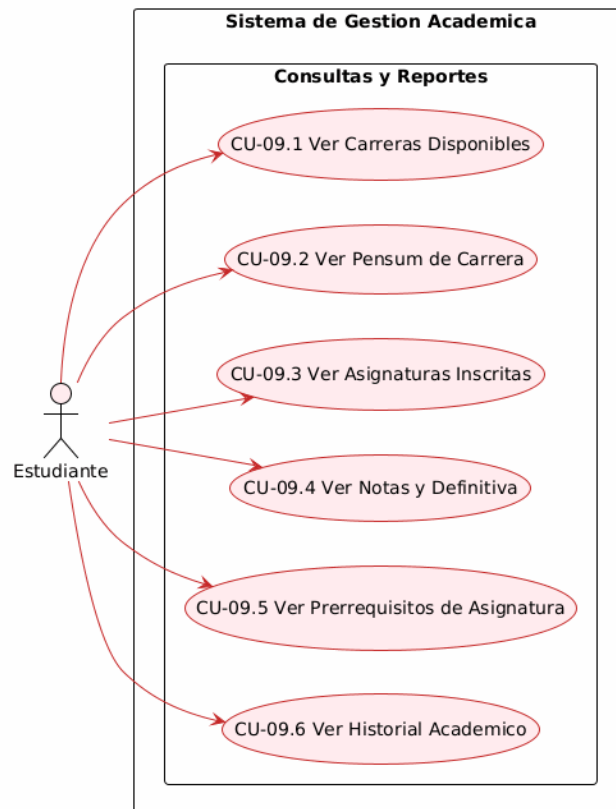


Figura 4: Casos de Uso del Estudiante

4.4 Detalle de Subsistemas

A continuacion se presentan los diagramas de casos de uso detallados por subsistema, incluyendo relaciones de inclusion («include») y extension («extend»).

4.4.1 Subsistema de Gestion de Carreras

Please use 'loption handwritten true' to enable handwritten

Subsistema de Gestion de Carreras - Casos de Uso Detallados

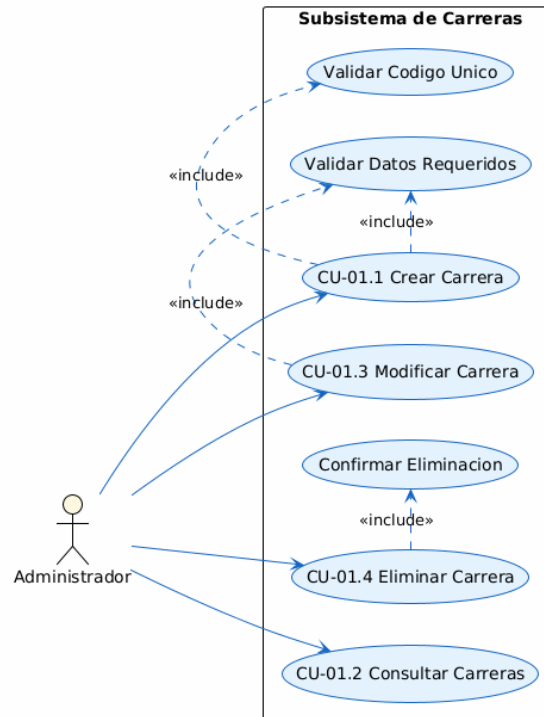


Figura 5: Subsistema de Gestion de Carreras

Descripcion de casos de uso:

- **CU-01.1 Crear Carrera:** Permite al administrador registrar una nueva carrera academica en el sistema. Incluye la validacion de codigo unico y datos requeridos.
- **CU-01.2 Consultar Carreras:** Permite visualizar el listado de carreras registradas con sus atributos.
- **CU-01.3 Modificar Carrera:** Permite actualizar la informacion de una carrera existente.
- **CU-01.4 Eliminar Carrera:** Permite eliminar una carrera previa confirmacion.

4.4.2 Subsistema de Gestion de Estudiantes

Please use 'loption handwritten true' to enable handwritten

Subsistema de Gestion de Estudiantes - Casos de Uso Detallados

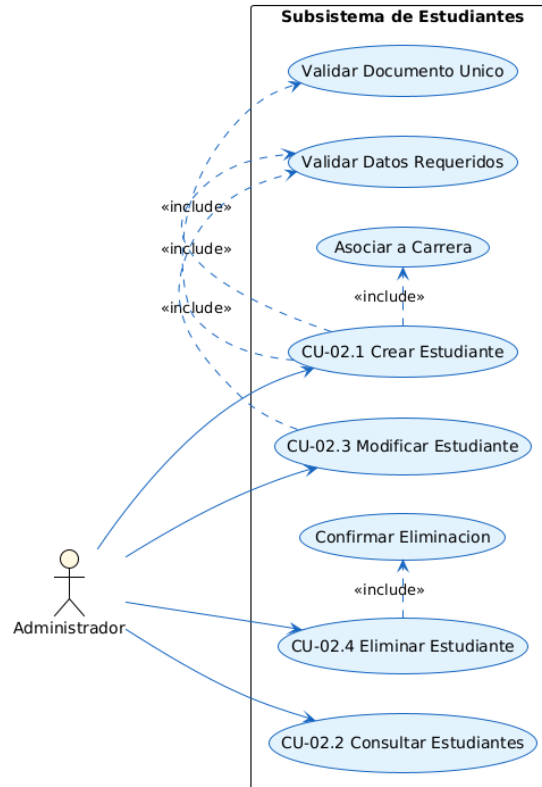


Figura 6: Subsistema de Gestion de Estudiantes

Descripcion de casos de uso:

- **CU-02.1 Crear Estudiante:** Registra un nuevo estudiante validando documento unico y asociandolo a una carrera.
- **CU-02.2 Consultar Estudiantes:** Visualiza el listado de estudiantes con filtros por carrera.
- **CU-02.3 Modificar Estudiante:** Actualiza datos personales y academicos del estudiante.
- **CU-02.4 Eliminar Estudiante:** Elimina un estudiante previa confirmacion.

4.4.3 Subsistema de Gestion de Asignaturas

Please use 'loption handwritten true' to enable handwritten

Subsistema de Gestion de Asignaturas - Casos de Uso Detallados

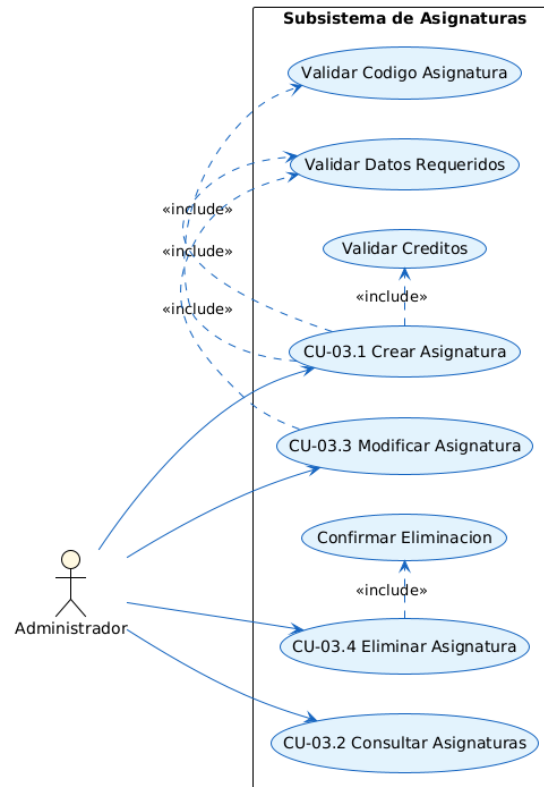


Figura 7: Subsistema de Gestion de Asignaturas

Descripcion de casos de uso:

- **CU-03.1 Crear Asignatura:** Registra una nueva asignatura con codigo, nombre y creditos.
- **CU-03.2 Consultar Asignaturas:** Lista todas las asignaturas del catalogo.
- **CU-03.3 Modificar Asignatura:** Actualiza atributos de una asignatura.
- **CU-03.4 Eliminar Asignatura:** Elimina una asignatura del catalogo.

4.4.4 Subsistema de Gestion de Profesores

Please use 'option handwritten true' to enable handwritten

Subsistema de Gestion de Profesores - Casos de Uso Detallados

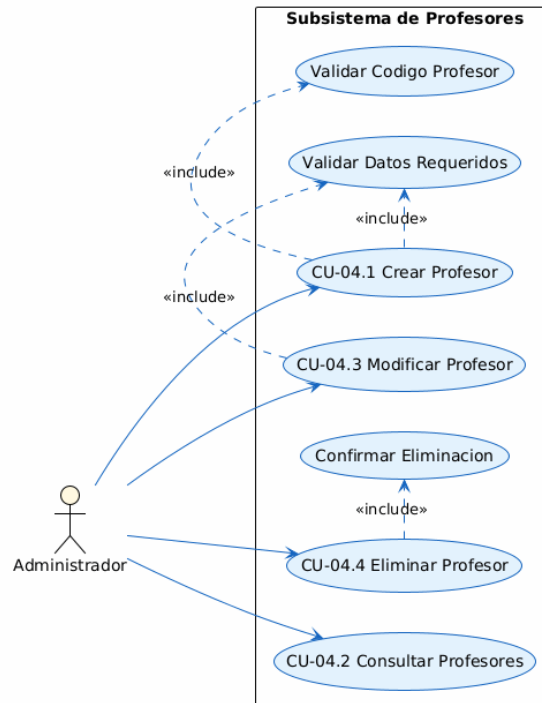


Figura 8: Subsistema de Gestion de Profesores

Descripcion de casos de uso:

- **CU-04.1 Crear Profesor:** Registra un nuevo profesor en el sistema.
- **CU-04.2 Consultar Profesores:** Visualiza el listado de profesores activos.
- **CU-04.3 Modificar Profesor:** Actualiza informacion del profesor.
- **CU-04.4 Eliminar Profesor:** Elimina un profesor del sistema.

4.4.5 Subsistema de Carga Academica

Please use '!option handwritten true' to enable handwritten

Subsistema de Carga Academica - Casos de Uso Detallados

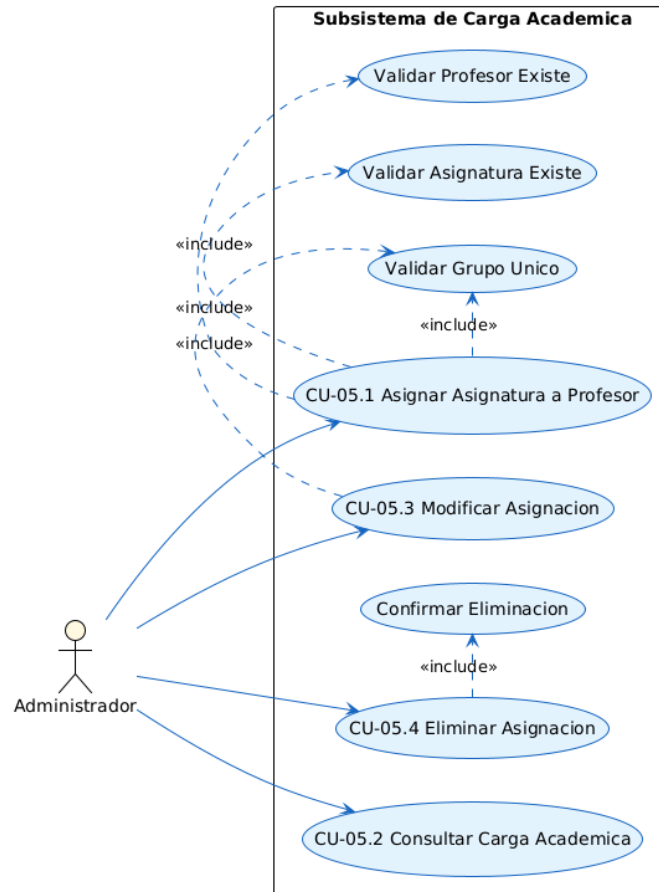


Figura 9: Subsistema de Carga Academica

Descripcion de casos de uso:

- **CU-05.1 Asignar Asignatura a Profesor:** Asocia una asignatura a un profesor para un grupo especifico.
- **CU-05.2 Consultar Carga Academica:** Visualiza las asignaciones actuales.
- **CU-05.3 Modificar Asignacion:** Cambia profesor o grupo de una asignacion.
- **CU-05.4 Eliminar Asignacion:** Elimina una asignacion de carga academica.

4.4.6 Subsistema de Inscripciones

Please use 'option handwritten true' to enable handwritten

Subsistema de Inscripciones - Casos de Uso Detallados

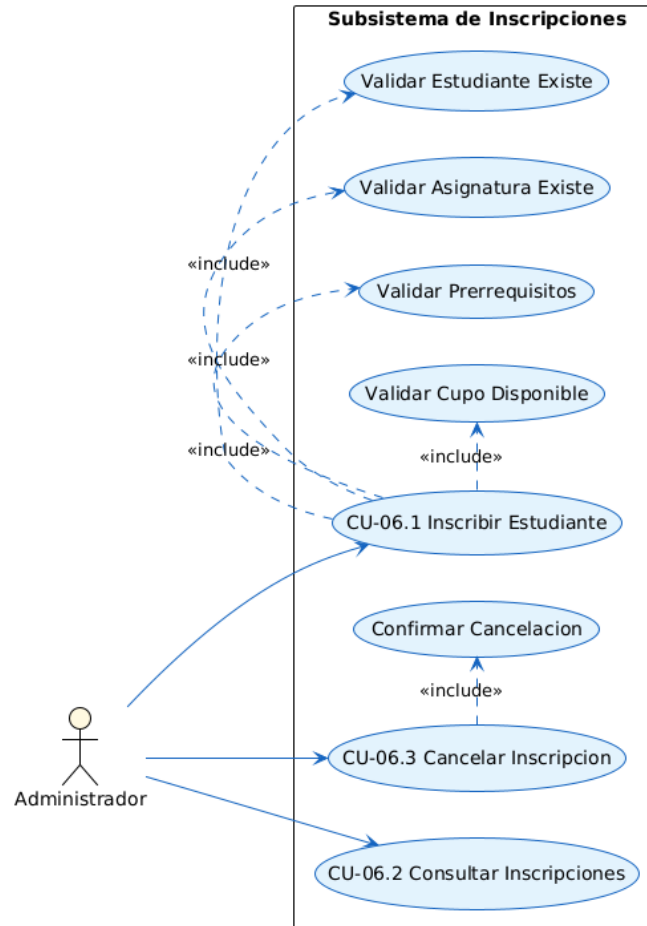


Figura 10: Subsistema de Inscripciones

Descripcion de casos de uso:

- **CU-06.1 Inscribir Estudiante:** Inscribe un estudiante a una asignatura validando prerrequisitos y cupo. Utiliza el stored procedure `sp_inscribir_estudiante`.
- **CU-06.2 Consultar Inscripciones:** Lista las inscripciones activas por periodo.
- **CU-06.3 Cancelar Inscripcion:** Cancela una inscripcion previa confirmacion.

4.4.7 Subsistema de Registro de Notas

Please use '!option handwritten true' to enable handwritten

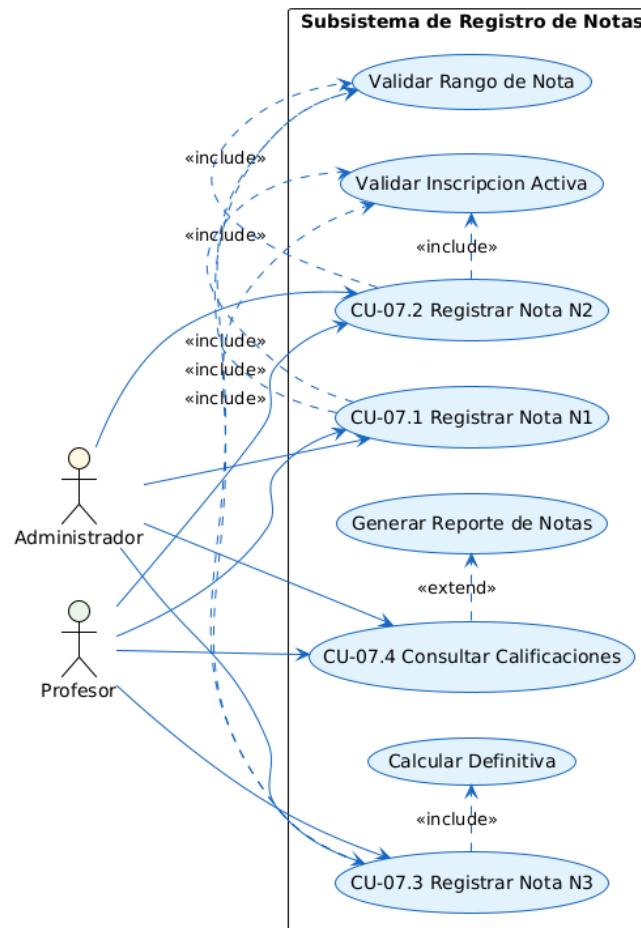
Subsistema de Registro de Notas - Casos de Uso Detallados

Figura 11: Subsistema de Registro de Notas

Descripcion de casos de uso:

- **CU-07.1 Registrar Nota N1:** Registra la nota del primer corte (35 %).
- **CU-07.2 Registrar Nota N2:** Registra la nota del segundo corte (35 %).
- **CU-07.3 Registrar Nota N3:** Registra la nota del tercer corte (30 %). El trigger `trg_calculo_notas` calcula automaticamente la definitiva.
- **CU-07.4 Consultar Calificaciones:** Visualiza las notas registradas y la definitiva calculada.

La formula de calculo de la nota definitiva es:

$$Definitiva = (n_1 \times 0,35) + (n_2 \times 0,35) + (n_3 \times 0,30) \quad (1)$$

4.4.8 Subsistema de Prerrequisitos

Please use 'option handwritten true' to enable handwritten

Subsistema de Prerrequisitos - Casos de Uso Detallados

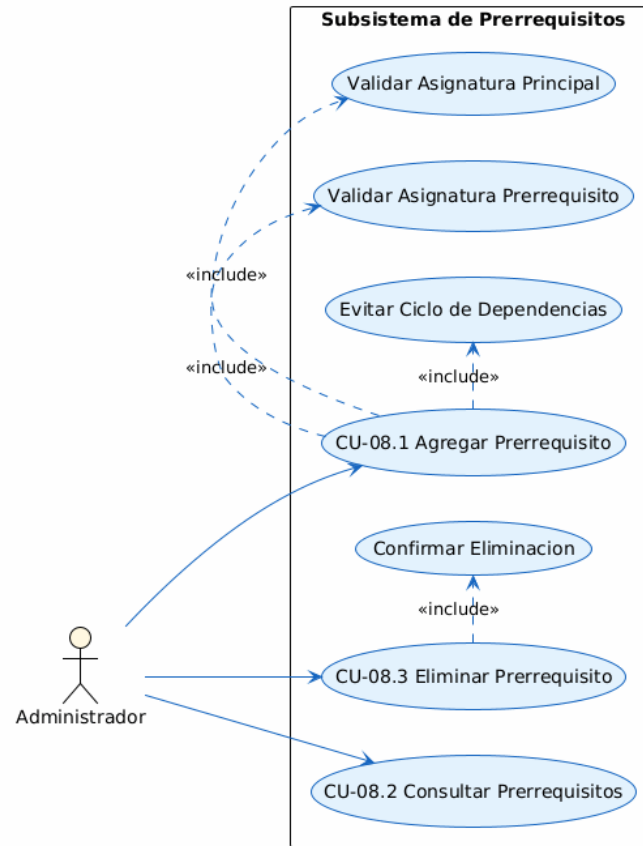


Figura 12: Subsistema de Prerrequisitos

Descripcion de casos de uso:

- **CU-08.1 Agregar Prerrequisito:** Define una asignatura como prerrequisito de otra, validando que no se creen ciclos de dependencia.
- **CU-08.2 Consultar Prerrequisitos:** Visualiza la cadena de prerrequisitos de una asignatura.
- **CU-08.3 Eliminar Prerrequisito:** Elimina una relacion de prerrequisito.

4.4.9 Subsistema de Gestion de Pensum

Please use 'loption handwritten true' to enable handwritten

Subsistema de Gestion de Pensum - Casos de Uso Detallados

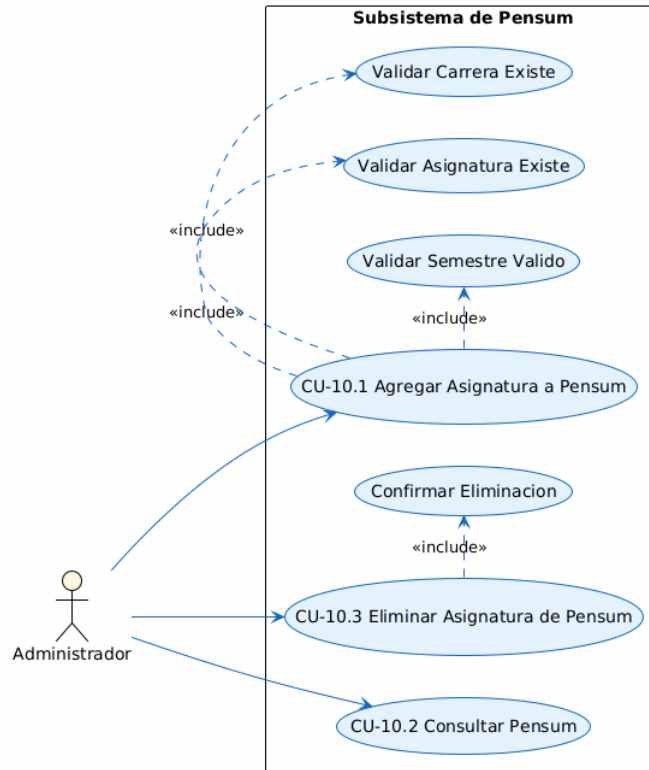


Figura 13: Subsistema de Gestion de Pensum

Descripcion de casos de uso:

- **CU-10.1 Agregar Asignatura a Pensum:** Asocia una asignatura a una carrera en un semestre especifico.
- **CU-10.2 Consultar Pensum:** Visualiza el pensum completo de una carrera organizado por semestres.
- **CU-10.3 Eliminar Asignatura de Pensum:** Elimina una asignatura del pensum de una carrera.

4.4.10 Subsistema de Consultas y Reportes

Please use '!option handwritten true' to enable handwritten

Subsistema de Consultas y Reportes - Casos de Uso Detallados

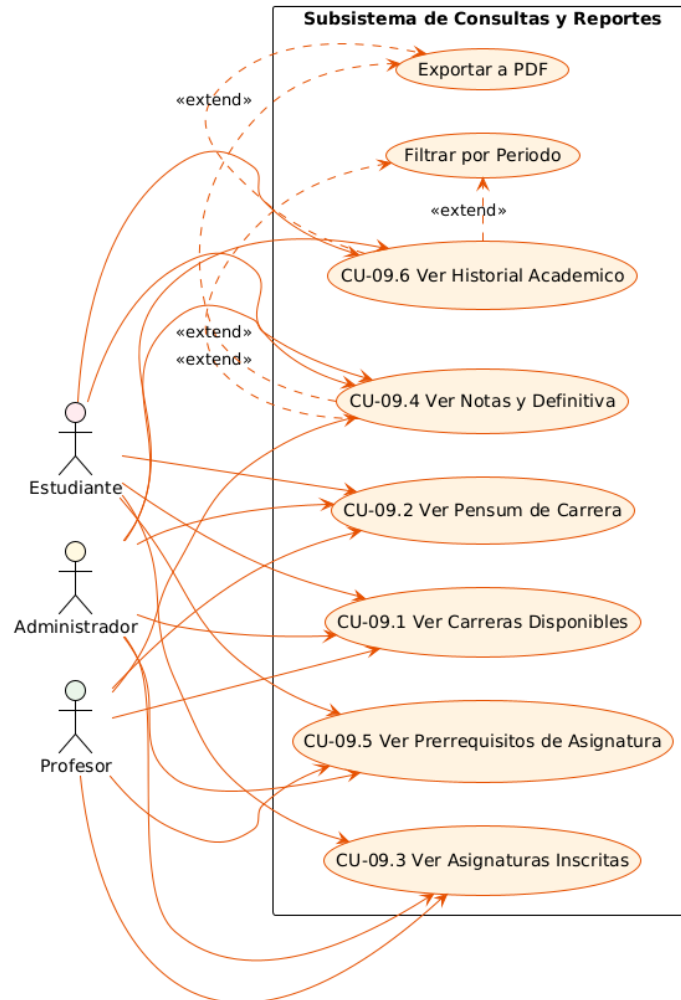


Figura 14: Subsistema de Consultas y Reportes

Descripcion de casos de uso:

- **CU-09.1 Ver Carreras Disponibles:** Lista las carreras activas del sistema.
- **CU-09.2 Ver Pensum de Carrera:** Muestra el pensum detallado de una carrera especifica.
- **CU-09.3 Ver Asignaturas Inscritas:** Lista las asignaturas en las que el estudiante esta inscrito.
- **CU-09.4 Ver Notas y Definitiva:** Muestra las calificaciones del estudiante con la nota definitiva.
- **CU-09.5 Ver Prerrequisitos de Asignatura:** Consulta los prerrequisitos de una asignatura especifica.

- **CU-09.6 Ver Historial Academico:** Muestra el historial completo de notas y asignaturas cursadas.

4.5 Matriz de Trazabilidad de Casos de Uso

La figura 15 presenta la matriz de trazabilidad que relaciona cada caso de uso con los actores que tienen permiso para ejecutarlo.

Please use '!option handwritten true' to enable handwritten

Matriz de Trazabilidad de Casos de Uso

Matriz			
Caso de Uso	Admin	Profesor	Estudiante
CU-01.1 Crear Carrera	●		
CU-01.2 Consultar Carreras	●		
CU-01.3 Modificar Carrera	●		
CU-01.4 Eliminar Carrera	●		
CU-02.1 Crear Estudiante	●		
CU-02.2 Consultar Estudiantes	●		
CU-02.3 Modificar Estudiante	●		
CU-02.4 Eliminar Estudiante	●		
CU-03.1 Crear Asignatura	●		
CU-03.2 Consultar Asignaturas	●		
CU-03.3 Modificar Asignatura	●		
CU-03.4 Eliminar Asignatura	●		
CU-04.1 Crear Profesor	●		
CU-04.2 Consultar Profesores	●		
CU-04.3 Modificar Profesor	●		
CU-04.4 Eliminar Profesor	●		
CU-05.1 Asignar Asignatura	●		
CU-05.2 Consultar Carga	●		
CU-05.3 Modificar Asignacion	●		
CU-05.4 Eliminar Asignacion	●		
CU-06.1 Inscribir Estudiante	●		
CU-06.2 Consultar Inscripciones	●		
CU-06.3 Cancelar Inscripcion	●		
CU-07.1 Registrar Nota N1	●	●	
CU-07.2 Registrar Nota N2	●	●	
CU-07.3 Registrar Nota N3	●	●	
CU-07.4 Consultar Calificaciones	●	●	
CU-08.1 Agregar Prerrequisito	●		
CU-08.2 Consultar Prerrequisitos	●		
CU-08.3 Eliminar Prerrequisito	●		
CU-09.1 Ver Carreras	●	●	●
CU-09.2 Ver Pensum	●	●	●
CU-09.3 Ver Asignaturas	●	●	●
CU-09.4 Ver Notas	●	●	●
CU-09.5 Ver Prerrequisitos	●	●	●
CU-09.6 Ver Historial	●		●
CU-10.1 Agregar a Pensum	●		
CU-10.2 Consultar Pensum	●		
CU-10.3 Eliminar de Pensum	●		

Figura 15: Matriz de Trazabilidad de Casos de Uso

Resumen de acceso por rol:

- **Administrador:** Acceso total a los 37 casos de uso del sistema.
- **Profesor:** Acceso a 9 casos de uso (registro de notas y consultas).
- **Estudiante:** Acceso a 6 casos de uso (consultas de solo lectura).

5 Vista Logica

5.1 Descripcion General

La vista logica describe la estructura estatica del sistema mediante clases, paquetes y sus relaciones. El sistema sigue una arquitectura en capas con separacion clara entre presentacion, control, modelo y acceso a datos.

5.2 Estructura de Paquetes

El codigo fuente Java esta organizado en los siguientes paquetes:

Listing 1: Estructura de paquetes del sistema

```
com.academico/
|-- config/           # Configuracion de conexion a BD (Singleton)
|-- controller/       # Controladores JavaFX (MVC)
|-- dao/              # Data Access Objects
|-- model/            # Entidades del dominio
|-- Main.java         # Punto de entrada
```

5.3 Descripcion de Clases Principales

5.3.1 Capa de Modelo (Model)

Las clases del modelo representan las entidades del dominio academico:

Clase	Descripcion y Atributos Principales
Carrera	Representa una carrera academica. Atributos: codigo, nombre, numero de semestres.
Estudiante	Representa un estudiante. Atributos: codigo, nombre, carrera (FK).
Asignatura	Representa una asignatura del catalogo. Atributos: codigo, nombre, creditos.
Profesor	Representa un profesor. Atributos: codigo, nombre, especialidad.
Imparte	Representa la asignacion de una asignatura a un profesor. Atributos: profesor, asignatura, grupo, anio, periodo.
Inscripcion	Representa la inscripcion de un estudiante. Atributos: estudiante, asignatura, notas (n1, n2, n3, definitiva).
Pensum	Relacion entre carrera y asignatura. Atributos: carrera, asignatura, semestre.
Prerrequisito	Relacion de dependencia entre asignaturas. Atributos: asignatura, prerrequisito.

5.3.2 Capa de Acceso a Datos (DAO)

Cada entidad del modelo cuenta con un DAO correspondiente que implementa las operaciones CRUD:

- **CarreraDAO**: Operaciones CRUD para carreras.
- **EstudianteDAO**: Operaciones CRUD para estudiantes.
- **AsignaturaDAO**: Operaciones CRUD para asignaturas.
- **ProfesorDAO**: Operaciones CRUD para profesores.
- **ImparteDAO**: Operaciones para asignar y gestionar carga academica.
- **InscripcionDAO**: Operaciones de inscripcion y registro de notas.
- **PensumDAO**: Operaciones de gestion de pensum.
- **PrerrequisitoDAO**: Operaciones de gestion de prerrequisitos.

5.3.3 Clase OperationResult

La clase `OperationResult` encapsula el resultado de cualquier operacion de base de datos:

Listing 2: Clase `OperationResult`

```
public class OperationResult<T> {
    private boolean success;
    private String message;
    private T data;

    public static <T> OperationResult<T> success(T data, String
        message) { ... }
    public static <T> OperationResult<T> error(String message) {
        ... }
}
```

Esta clase permite propagar mensajes de exito o error desde la base de datos hasta la interfaz de usuario de forma uniforme.

5.4 Patrones de Diseno Aplicados

5.4.1 Patron Singleton

La clase `DatabaseConfig` implementa el patron Singleton para garantizar una unica conexion a la base de datos:

Listing 3: Implementacion del patron Singleton

```
public class DatabaseConfig {
    private static DatabaseConfig instance;
    private Connection connection;

    private DatabaseConfig() { ... }

    public static synchronized DatabaseConfig getInstance() {
        if (instance == null) {
            instance = new DatabaseConfig();
        }
    }
}
```

```
    }  
    return instance;  
}  
}
```

5.4.2 Patron DAO

Cada DAO sigue una estructura consistente. Ejemplo de CarreraDAO:

Listing 4: Patron DAO - CarreraDAO

```
public class CarreraDAO {  
    private Connection connection;  
  
    public OperationResult<Carrera> crear(Carrera carrera) { ... }  
    public OperationResult<List<Carrera>> consultarTodas() { ... }  
    public OperationResult<Carrera> consultarPorCodigo(String  
        codigo) { ... }  
    public OperationResult<Void> modificar(Carrera carrera) { ... }  
    public OperationResult<Void> eliminar(String codigo) { ... }  
}
```

6 Vista de Procesos

6.1 Descripcion General

La vista de procesos describe la dinamica del sistema, mostrando como los componentes interactuan en tiempo de ejecucion para cumplir con las funcionalidades definidas en los casos de uso.

6.2 Flujo de Inscripcion de Estudiante

El flujo de inscripcion es uno de los mas criticos del sistema, ya que involucra validaciones de prerequisites mediante el stored procedure `sp_inscribir_estudiante`:

1. El administrador selecciona el estudiante y la asignatura a inscribir.
2. El sistema invoca `sp_inscribir_estudiante(estudiante, asignatura)`.
3. El stored procedure verifica que el estudiante exista.
4. El SP verifica que la asignatura exista.
5. El SP valida que el estudiante haya aprobado todos los prerequisites.
6. Si la validacion falla, lanza `RAISE EXCEPTION` con mensaje descriptivo.
7. Si la validacion es exitosa, inserta el registro en la tabla `Inscribe`.
8. El trigger `trg_calculo_notas` inicializa las notas en `NULL`.
9. El sistema recibe el mensaje de exito y lo muestra al usuario.

6.3 Flujo de Registro de Notas

El registro de notas dispara automaticamente el calculo de la nota definitiva:

1. El profesor o administrador ingresa la nota N1, N2 o N3.
2. El sistema ejecuta `UPDATE Inscribe SET nX = valor`.
3. El trigger `trg_calculo_notas` se activa despues del `UPDATE`.
4. El trigger calcula: `definitiva = (n1 * 0.35) + (n2 * 0.35) + (n3 * 0.30)`.
5. El trigger actualiza el campo definitiva automaticamente.
6. El sistema confirma la operacion al usuario.

6.4 Flujo de Autenticacion

1. El usuario selecciona su rol (Administrador, Profesor o Estudiante).
2. El sistema establece la conexion a PostgreSQL con las credenciales correspondientes:
 - `app_admin / Admin_Secret2026*`
 - `app_profesor / Prof_Secret2026*`
 - `app_estudiante / Est_Secret2026*`
3. PostgreSQL verifica el rol y aplica los permisos RBAC correspondientes.
4. El sistema carga la interfaz acorde al rol autenticado.

7 Vista de Despliegue

7.1 Descripcion de la Infraestructura

El sistema se despliega en una arquitectura de dos capas:

1. **Capa de Cliente:** Aplicacion de escritorio JavaFX ejecutandose en la maquina del usuario.
2. **Capa de Datos:** Base de datos PostgreSQL 15 ejecutandose en un contenedor Docker.

7.2 Configuracion de Hardware y Software

Nodo	Hardware Minimo	Software Requerido
Cliente	CPU: 2 cores, RAM: 4 GB, Disco: 1 GB	Java 17, JavaFX, Maven 3.8
Servidor BD	CPU: 2 cores, RAM: 2 GB, Disco: 10 GB	Docker, Docker Compose, PostgreSQL 15

7.3 Diagrama de Despliegue

La configuracion de despliegue se describe textualmente dado que es una arquitectura simple de dos nodos:

- **Nodo Cliente:** Ejecuta la aplicacion JavaFX. Se comunica con el servidor de base de datos mediante JDBC a traves del puerto 5432.
- **Nodo Servidor PostgreSQL:** Contenedor Docker que expone el puerto 5432. Incluye scripts de inicializacion en `init-scripts/` que crean el esquema, tablas, triggers, stored procedures, roles y datos de prueba.

7.4 Configuracion de Red

- **Protocolo:** TCP/IP
- **Puerto de BD:** 5432
- **Driver JDBC:** PostgreSQL JDBC 42.7.3
- **Host:** localhost (o IP del contenedor Docker)

8 Vista de Datos

8.1 Modelo Entidad-Relacion

La base de datos del sistema sigue el siguiente modelo relacional:

Tabla	Descripcion y Atributos
Carreras	codigo (PK), nombre, num_semestres
Estudiantes	codigo (PK), nombre, cod_carrera (FK)
Asignaturas	codigo (PK), nombre, creditos
Profesores	codigo (PK), nombre, especialidad
Imparte	codigo (PK), cod_profesor (FK), cod_asignatura (FK), grupo, anio, periodo
Inscribe	codigo (PK), cod_estudiante (FK), cod_asignatura (FK), n1, n2, n3, definitiva
Incluye	codigo (PK), cod_carrera (FK), cod_asignatura (FK), semestre
Requiere	codigo (PK), cod_asignatura (FK), cod_prerrequisito (FK)

8.2 Restricciones de Integridad

- **Claves primarias:** Todas las tablas tienen un campo codigo como clave primaria.
- **Claves foraneas:** Las relaciones entre tablas garantizan la integridad referencial.
- **Regla de negocio:** La nota definitiva se calcula automaticamente mediante trigger.
- **Validacion:** Los prerequisitos se validan mediante stored procedure antes de la inscripcion.

8.3 Triggers y Stored Procedures

8.3.1 Trigger trg_calculo_notas

Listing 5: Trigger de calculo de notas definitivas

```
CREATE TRIGGER trg_calculo_notas
AFTER INSERT OR UPDATE OF n1, n2, n3 ON Inscribe
FOR EACH ROW
EXECUTE FUNCTION calcular_definitiva();
```

8.3.2 Stored Procedure sp_inscribir_estudiante

Listing 6: Stored Procedure de validacion de inscripcion

```
CREATE OR REPLACE PROCEDURE sp_inscribir_estudiante(
    p_estudiante VARCHAR,
    p_asignatura VARCHAR
)
LANGUAGE plpgsql
```

```

AS $$
BEGIN
    -- Validar prerrequisitos
    IF EXISTS (
        SELECT 1 FROM Requiere r
        WHERE r.cod_asignatura = p_asignatura
        AND r.cod_prerrequisito NOT IN (
            SELECT i.cod_asignatura FROM Inscribe i
            WHERE i.cod_estudiante = p_estudiante
            AND i.definitiva >= 3.0
        )
    ) THEN
        RAISE EXCEPTION 'El estudiante no ha aprobado todos los
            prerrequisitos';
    END IF;

    -- Realizar inscripcion
    INSERT INTO Inscribe (cod_estudiante, cod_asignatura, n1, n2,
        n3)
    VALUES (p_estudiante, p_asignatura, NULL, NULL, NULL);
END;
$$;

```

8.4 Control de Acceso (RBAC)

El sistema implementa tres roles de PostgreSQL con permisos diferenciados:

Rol	Privilegios
rol_admin_academico	Control total (SELECT, INSERT, UPDATE, DELETE) sobre todas las tablas.
rol_profesor	SELECT en todas las tablas, UPDATE solo en Inscribe (notas).
rol_estudiante	SELECT en Carreras, Asignaturas, Incluye, Requiere (solo lectura).

9 Tamano y Rendimiento

9.1 Estimacion de Tamano

Componente	Estimacion	Observaciones
Lineas de codigo Java	Aprox. 5,000	Incluye modelo, DAO, controladores y utilidades
Lineas de codigo SQL	Aprox. 1,500	DDL, DML, triggers, SP y roles
Numero de clases Java	25-30	Incluye entidades, DAOs y controladores
Numero de tablas BD	8	Mas tablas de auditoria si se requieren

9.2 Requisitos de Rendimiento

- **Tiempo de inicio:** La aplicacion debe iniciar en menos de 5 segundos.
- **Tiempo de respuesta de consultas:** Las consultas basicas deben responder en menos de 2 segundos para hasta 10,000 registros.
- **Concurrencia:** El sistema soporta multiples usuarios conectados simultaneamente mediante el pool de conexiones de PostgreSQL.

10 Calidad

10.1 Mantenibilidad

- **Modularidad:** El sistema esta organizado en capas independientes que pueden modificarse sin afectar las demas.
- **Reutilizacion:** Los DAOs siguen una estructura consistente que facilita la creacion de nuevos DAOs.
- **Documentacion:** El codigo esta documentado con Javadoc y los diagramas UML describen la arquitectura.

10.2 Portabilidad

- La aplicacion Java es multiplataforma (Windows, Linux, macOS) gracias a la JVM.
- La base de datos esta contenedorizada con Docker, facilitando su despliegue en cualquier sistema operativo.

10.3 Seguridad

- **Autenticacion:** Basada en credenciales de PostgreSQL con roles diferenciados.
- **Autorizacion:** RBAC a nivel de base de datos que restringe operaciones segun el rol.
- **Validacion:** Las validaciones de negocio ocurren en la base de datos, evitando bypass desde la aplicacion.

10.4 Usabilidad

- Interfaz grafica intuitiva desarrollada en JavaFX.
- Mensajes de error claros provenientes directamente de PostgreSQL.
- Flujo de trabajo logico que guia al usuario en cada operacion.

10.5 Fiabilidad

- **Integridad transaccional:** Las operaciones criticas (inscripciones, notas) estan protegidas por transacciones.
- **Propagacion de errores:** La clase OperationResult garantiza que los errores de base de datos lleguen al usuario.
- **Consistencia de datos:** Los triggers mantienen la consistencia automatica de las notas definitivas.

11 Anexo:Codigo Fuente PlantUML

Este anexo contiene el codigo fuente PlantUML de los diagramas de casos de uso presentados en este documento. El profesor puede verificar y modificar estos diagramas usando cualquier herramienta compatible con PlantUML.

11.1 Diagrama General de Casos de Uso

El archivo fuente completo se encuentra en:

```
docs/diagramas/plantuml/01_diagrama_general_casos_uso.puml
```

11.2 Casos de Uso del Administrador

El archivo fuente completo se encuentra en:

```
docs/diagramas/plantuml/02_casos_uso_administrador.puml
```

11.3 Casos de Uso del Profesor

El archivo fuente completo se encuentra en:

```
docs/diagramas/plantuml/03_casos_uso_profesor.puml
```

11.4 Casos de Uso del Estudiante

El archivo fuente completo se encuentra en:

```
docs/diagramas/plantuml/04_casos_uso_estudiante.puml
```

11.5 Diagramas de Subsistemas

Todos los diagramas detallados por subsistema se encuentran en:

```
docs/diagramas/plantuml/05_subsistema_carreras.puml
docs/diagramas/plantuml/06_subsistema_estudiantes.puml
docs/diagramas/plantuml/07_subsistema_asignaturas.puml
docs/diagramas/plantuml/08_subsistema_profesores.puml
docs/diagramas/plantuml/09_subsistema_carga_academica.puml
docs/diagramas/plantuml/10_subsistema_inscripciones.puml
docs/diagramas/plantuml/11_subsistema_notas.puml
docs/diagramas/plantuml/12_subsistema_prerrequisitos.puml
docs/diagramas/plantuml/13_subsistema_pensum.puml
docs/diagramas/plantuml/14_subsistema_consultas.puml
```

11.6 Matriz de Trazabilidad

El archivo fuente completo se encuentra en:

```
docs/diagramas/plantuml/15_matriz_trazabilidad.puml
```